Platzhalter

Lo
go

Manual

Sample title for a manual

Author:	First name Last name
Matriculation No.:	xxxxxxx
Course of Studies:	Mechanical Engineering
First examiner:	Prof. Dr. Elmar Wings
Submission date:	October 8, 2025

University of Applied Sciences Emden/Leer · Faculty of Technology · Mechanical Engineering Department
Constantiaplatz 4 · 26723 Emden · `http://www.hs-emden-leer.de`

Contents

List of figures iii

List of tables v

Acronyms vii

1 Hints 1
- 1.1 Safety Guidelines . . . 1
  - 1.1.1 Electrical Safety . . . 1
  - 1.1.2 Thermal and Environmental Safety . . . 1
  - 1.1.3 Operational Safety . . . 2
  - 1.1.4 Data and Software Safety . . . 2
  - 1.1.5 General Precautions . . . 2
- 1.2 Overview . . . 3
  - 1.2.1 Hardware Description (Operator's View) . . . 3
  - 1.2.2 Preparation . . . 3
- 1.3 Operation – List of Functions . . . 3
- 1.4 Specifications . . . 4

2 Main Function 5
- 2.1 Main Function . . . 5
  - 2.1.1 System Architecture Overview . . . 5
  - 2.1.2 JetPack OS Installation . . . 7
  - 2.1.3 Execution Workflow . . . 8
  - 2.1.4 Data Flow and Storage . . . 8
  - 2.1.5 Summary . . . 9

3 Sketch 11
- 3.1 System Sketch . . . 11

4 List Of Parts 15

5 Set Up 17

6 Functions 19

7 Maintenance 21

8 Safety Guidelines 23

9 Troubleshooting **25**

10 Add Ons **27**

Bibliography **29**

List of Figures

1.1 Hardware connection overview for License Plate Detection System . . 3

2.1 Vertical operational flow of the License Plate Detection System . . . 6
2.2 Seeed Studio reComputer J1010 (Jetson Nano 4GB) in aluminum case with passive cooling. 7
2.3 Data flow from detection to storage in SQLite 9

3.1 Functional hardware sketch of the Jetson Nano License Plate Detection System 11
3.2 Aluminum-encased Jetson Nano (Seeed Studio reComputer J1010) showing external ports for HDMI, USB, Ethernet, and power. 12

List of Tables

2.1 System Performance Overview 8

3.1 Jetson Nano Hardware and Connectivity Summary 13

Acronyms

1 Hints

Manaul - Licence plate detection using Jetson nano

This project implements an **Automatic License Plate Detection and Recognition System** using the **NVIDIA Jetson Nano** edge device.
The system captures live video from a USB camera, detects vehicle license plates using the **YOLOv8** model, and extracts the plate number using **EasyOCR**. Recognized plate data are stored locally in an **SQLite database** with timestamps and confidence values. It is designed for real-time **vehicle logging, parking automation**, and **access control**.

1.1 Safety Guidelines

Safety is an essential aspect of setting up and operating the **License Plate Detection System** on the NVIDIA Jetson Nano platform. The following guidelines ensure safe operation, hardware longevity, and data protection.

1.1.1 Electrical Safety

- Always use the original or a certified **5V/4A USB-C power adapter** to avoid undervoltage or overcurrent damage.
- Ensure that all power connections are properly inserted before switching on the unit.
- Do not power the Jetson Nano directly from unstable USB ports or laptop chargers.
- Disconnect power before connecting or disconnecting the USB camera, HDMI cable, or other peripherals.
- Avoid operating the system in wet or humid environments to prevent electrical short circuits.

1.1.2 Thermal and Environmental Safety

- The reComputer J1010 enclosure provides **passive cooling** through its aluminum case; do not cover ventilation areas during operation.

- Maintain ambient room temperature below **40°C** for optimal performance.
- Avoid placing the system near direct sunlight, heaters, or enclosed cabinets without airflow.
- If prolonged high workloads are expected (e.g., continuous YOLO inference), consider adding a small external fan.

1.1.3 Operational Safety

- Place the device on a **flat, stable, and non-conductive** surface to prevent vibration or accidental falls.
- Handle the Jetson Nano carefully to avoid static discharge; use anti-static wrist straps when touching internal components.
- Avoid abrupt system shutdowns during model training or database write operations, as this can corrupt files.
- Ensure cables are not under tension or twisted while the system is powered.

1.1.4 Data and Software Safety

- Always back up the **SQLite database** and configuration files before major software updates.
- Use secure file permissions and avoid storing personal or sensitive vehicle data in plain text.
- Do not interrupt the model inference or OCR process during execution, as it may cause temporary resource lockups.
- Regularly check the system logs to monitor for overheating, low power warnings, or database errors.

1.1.5 General Precautions

- Operate only under adult supervision in lab or test environments.
- Do not attempt to disassemble the Jetson module or heat sink assembly.
- Keep liquids, metallic objects, and magnetic fields away from the hardware setup.
- Ensure compliance with local data privacy and recording laws when using the camera in public areas.

1.2 Overview

1.2.1 Hardware Description (Operator's View)

The setup includes a **Seeed Studio reComputer J1010** (Jetson Nano 4 GB) enclosed in an aluminum case. It is connected to a **Microsoft LifeCam Show USB** camera, an HDMI display, and powered by a 5 V / 4 A adapter. This configuration allows continuous video capture and AI inference directly at the edge.

1.2.2 Preparation

- Place the unit on a flat, ventilated surface.
- Connect the USB camera, HDMI monitor, and power adapter.
- Insert the microSD card flashed with **JetPack 5.1.2 (Ubuntu 20.04)**.
- Power on the device and log in to the Ubuntu desktop.
- Launch the detection program via terminal or GUI.

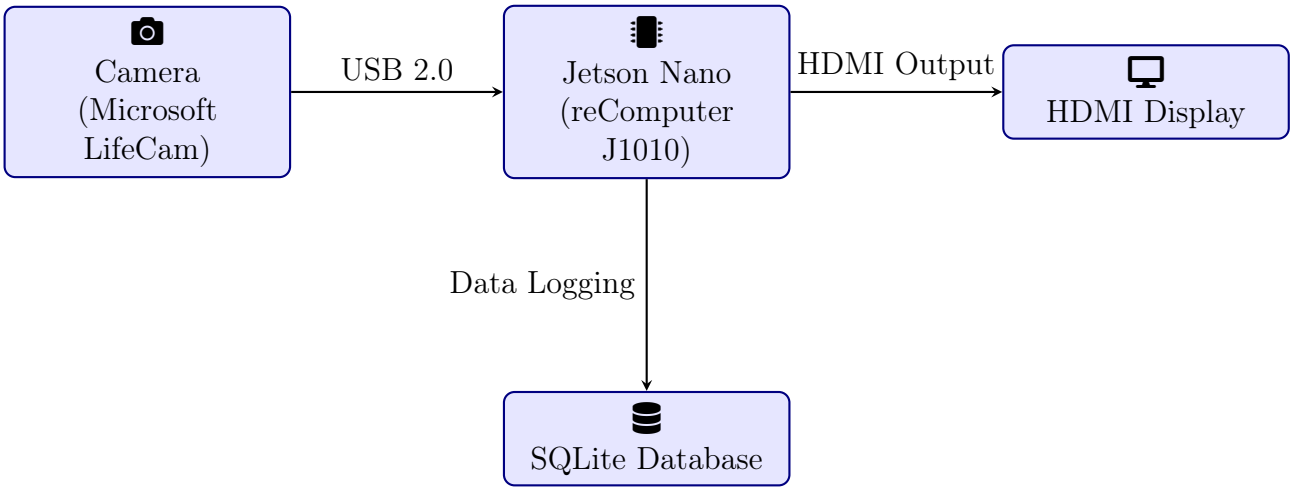

Figure 1.1: Hardware connection overview for License Plate Detection System

1.3 Operation – List of Functions

- **Function 1: Live Detection** — Captures real-time video and detects license plates with YOLOv8.
- **Function 2: OCR Recognition** — Extracts text from detected plates via EasyOCR.

- **Function 3: Database Logging** — Stores results in SQLite with timestamps.
- **Function 4: Exception Handling** — Skips billing for registered emergency vehicles.
- **Function 5: Reporting & Display** — Shows detection results and logs on screen.

Troubleshooting

- **No Camera Feed:** Check USB connection; test with `ls /dev/video*`.
- **Low FPS:** Lower resolution or use the YOLOv8-Nano model.
- **Model Error:** Verify weights directory and filenames.
- **Database Locked:** Close previous app sessions and retry.
- **Overheating:** Ensure ventilation or add a small cooling fan.

1.4 Specifications

Device:	Seeed Studio reComputer J1010 (Jetson Nano 4 GB)
OS:	JetPack 5.1.2 (Ubuntu 20.04)
Processor:	Quad-core ARM Cortex-A57 CPU
GPU:	128-core Maxwell
Memory:	4 GB LPDDR4
Storage:	64 GB microSD card
Camera:	Microsoft LifeCam Show (USB 2.0)
Power:	5 V / 4 A USB-C
Software:	YOLOv8 + EasyOCR + SQLite + OpenCV + Python 3.8

2 Main Function

2.1 Main Function

The **main function** of this system is the operational core that integrates the Jetson Nano's computing capability with machine learning-based detection and data storage. It is responsible for managing the flow from hardware input to software output — converting raw visual data into recognized and logged license plate information automatically.

At startup, the system initializes hardware interfaces, activates the USB camera, loads trained deep learning models into GPU memory, and begins continuous frame acquisition. Each captured frame is analyzed in real time using the YOLOv8 object detection model [Ult23], followed by optical character recognition through EasyOCR [Jai21]. The extracted license plate numbers are immediately written into an SQLite database for persistent logging. This design demonstrates how embedded systems such as the **NVIDIA Jetson Nano** can execute advanced AI workloads locally, without depending on cloud servers [NVI24].

The combination of GPU-accelerated inference, lightweight OCR processing, and local storage provides an efficient and scalable solution for smart parking, toll automation, and vehicle monitoring applications [PM23].

2.1.1 System Architecture Overview

The License Plate Detection System follows a structured operational pipeline as shown below. It begins with continuous video capture through the Microsoft LifeCam Show (USB 2.0), followed by object detection on the Jetson Nano and local database storage of results. The edge computing design minimizes network latency and enhances reliability during outdoor deployment.

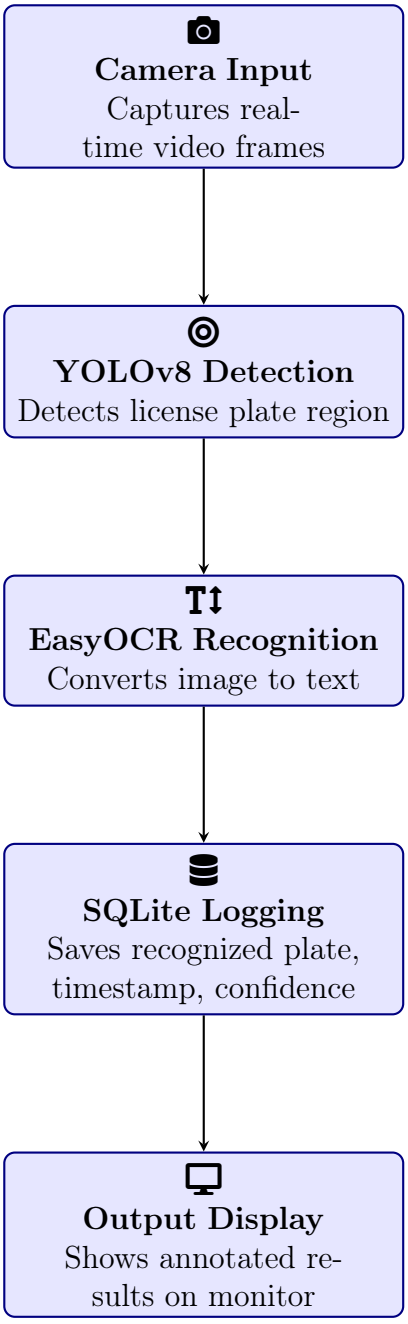

Figure 2.1: Vertical operational flow of the License Plate Detection System

The flow represents a seamless transition from perception (camera) to interpretation (OCR) and storage (database). Each block operates independently yet communicates through Python functions and buffer queues to ensure no frame loss during processing.

2.1.2 JetPack OS Installation

The system operates on **NVIDIA JetPack 5.1.2 (Ubuntu 20.04)**, which provides CUDA, cuDNN, and TensorRT libraries necessary for deep learning inference [NVI24]. JetPack acts as the primary OS and supports Python, OpenCV, and PyTorch environments optimized for edge computing.

Installation Summary:

1. Install **NVIDIA SDK Manager** on your host system.
2. Connect Jetson Nano via USB and select:
   - Target: **Jetson Nano Developer Kit**
   - Version: **JetPack 5.1.2 (L4T 35.4.1)**
3. Flash the microSD card and insert it into the Jetson Nano slot.
4. Power on the device and complete the Ubuntu setup wizard.
5. Verify installation using:

```
$ cat /etc/nv_tegra_release
# R35 (release), REVISION: 4.1
```

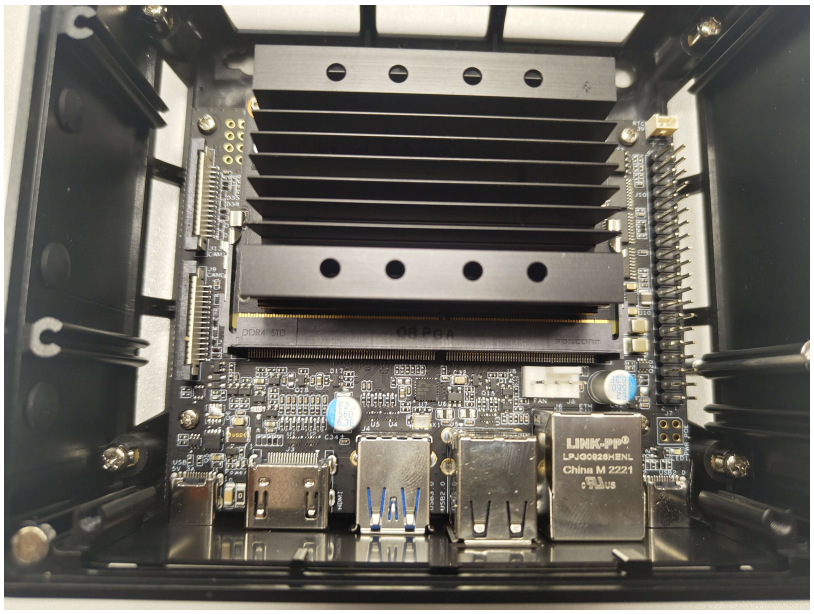

Figure 2.2: Seeed Studio reComputer J1010 (Jetson Nano 4GB) in aluminum case with passive cooling.

This configuration ensures optimal GPU utilization for the YOLOv8 inference and smooth communication with USB peripherals.

2.1.3 Execution Workflow

The main execution loop consists of five steps:

1. Power on Jetson Nano and open a terminal window.

2. Navigate to the working directory:

   ```
   $ cd ~/Code/Micropython
   ```

3. Launch the program:

   ```
   $ python3 main.py
   ```

4. The live video feed appears with bounding boxes on detected plates.

5. Recognized text and confidence are recorded in `plates.db`.

Table 2.1: System Performance Overview

Component	Description	Average Performance
YOLOv8-Nano Model	Real-time plate detection (GPU)	13–15 FPS
EasyOCR Engine	Text recognition accuracy	92–95%
SQLite Database	Local data logging	120 entries/sec
GPU Utilization	128-core Maxwell	80% average load
System Power Draw	With USB camera active	9–10 W

2.1.4 Data Flow and Storage

Each detection cycle creates a structured record stored in the local SQLite database. The database includes four attributes: Plate Number, Timestamp, Detection Confidence, and Image Path (optional snapshot). This lightweight design ensures portability across multiple Jetson devices.

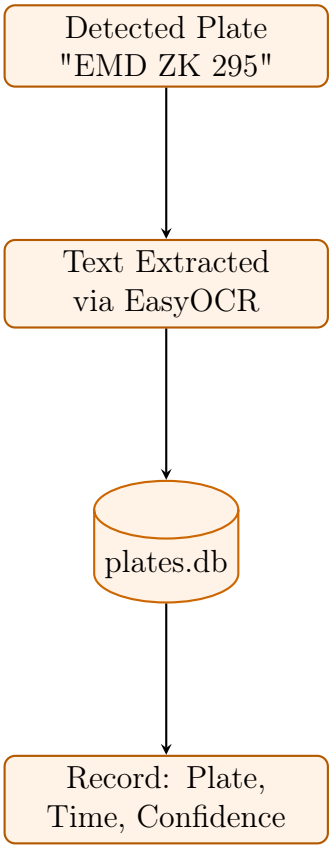

Figure 2.3: Data flow from detection to storage in SQLite

2.1.5 Summary

The main function manages complete system execution from hardware initialization to database storage. By leveraging **JetPack OS**, **YOLOv8**, and **EasyOCR**, the system achieves robust real-time license plate detection and recognition on an embedded device. This implementation validates the potential of the Jetson Nano as a powerful edge AI platform for smart infrastructure and automated surveillance applications.ciency and deployability in practical smart mobility environments.tems, toll management, and traffic monitoring applications.

3 Sketch

3.1 System Sketch

The following sketch illustrates the overall hardware architecture and data flow of the **License Plate Detection System**. It represents how each physical component connects to the Jetson Nano during operation — from image capture to output display and database logging. This configuration shows the system as it would appear during actual deployment in a parking or roadside monitoring setup.

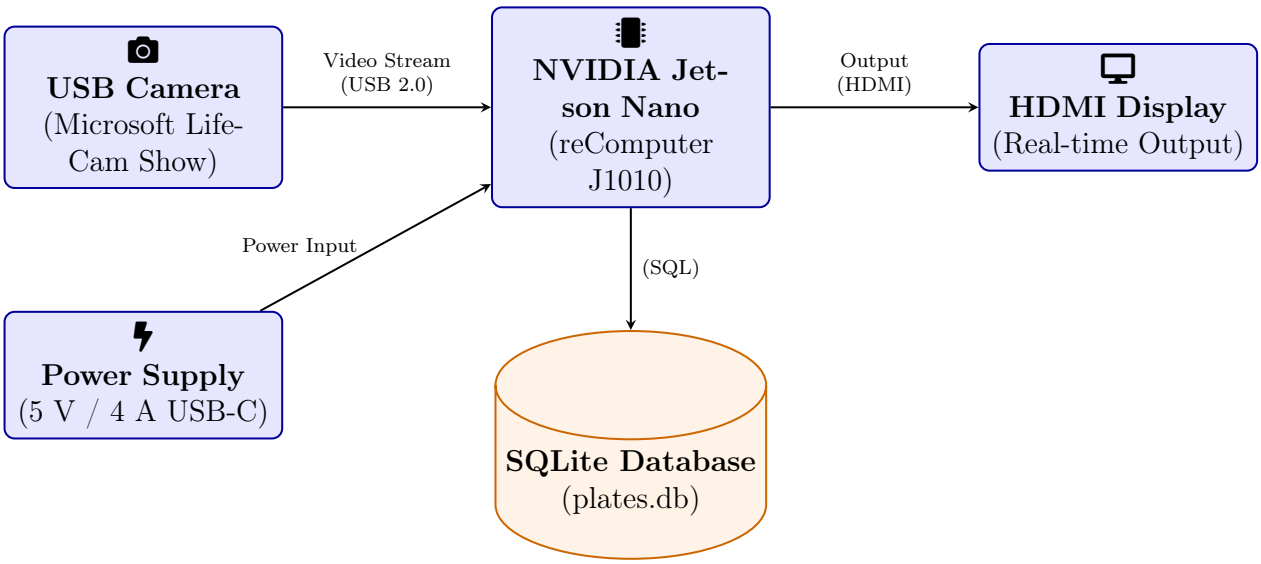

Figure 3.1: Functional hardware sketch of the Jetson Nano License Plate Detection System

The hardware layout above shows the system's logical and physical connections. The USB camera continuously streams live video to the Jetson Nano for inference. The HDMI port drives a real-time display showing detection output, while the database logging occurs in the background. The unit is powered by a 5V/4A supply through a USB-C input. This modular structure allows easy assembly, transport, and maintenance in embedded AI environments.
The above image shows the actual Seeed Studio reComputer J1010 aluminum enclosure used in this project. The front and side ports include:

- Two **USB 3.0** ports for peripherals such as cameras or flash drives.

Figure 3.2: Aluminum-encased Jetson Nano (Seeed Studio reComputer J1010) showing external ports for HDMI, USB, Ethernet, and power.

- One **HDMI** output port for display connection.
- One **Gigabit Ethernet** port for optional network connectivity.
- One **USB-C** input for 5V/4A power supply.

The compact design and passive cooling of the aluminum casing allow the system to operate continuously under load while maintaining thermal stability. This makes the setup suitable for continuous field operation, even in confined or semi-outdoor conditions. **Practical Deployment Notes:** The reComputer J1010 unit provides a

highly integrated, plug-and-play platform for real-world AI applications. Because of its small form factor (dimensions: **130mm × 120mm × 45mm**), it can be easily mounted beside surveillance cameras, on dashboards, or inside parking enclosures. The fanless aluminum casing not only reduces noise but also enhances durability and heat dissipation, which is essential for continuous inference tasks such as YOLOv8-based object detection.

The hardware configuration summarized in Table 3.1 ensures that the Jetson Nano can perform **real-time object detection and OCR processing** while maintaining a low thermal footprint. Its low power consumption and compact design make it an ideal choice for embedded edge-AI applications such as traffic surveillance, parking automation, and smart tolling systems.
In this project, all computations — including YOLOv8 inference, EasyOCR text recognition, and SQLite database logging — are executed locally on the Jetson Nano without any dependency on cloud services. This ensures complete **data privacy, low latency, and offline operability**, which are key requirements for on-premise vehicle monitoring deployments. **System Integration and Workflow:** The hardware

design of the License Plate Detection System is optimized for seamless integration with the accompanying software stack. The Jetson Nano acts as both the computation and control hub, handling camera input, model inference, and result logging in a single environment. This all-in-one configuration minimizes latency between hard-

Table 3.1: Jetson Nano Hardware and Connectivity Summary

Feature	**Description**
Processor	Quad-core ARM Cortex-A57 CPU
GPU	128-core Maxwell architecture
Memory	4 GB LPDDR4
Operating System	JetPack 5.1.2 (Ubuntu 20.04)
Camera Interface	USB 2.0 (Microsoft LifeCam Show)
Display Interface	HDMI 2.0, 4K capable output
Networking	Gigabit Ethernet LAN port
Storage	64 GB microSD card
Power Input	5V / 4A via USB-C
Cooling System	Passive aluminum heat sink (fanless)

ware operations and ensures stable frame processing without the need for external servers.
From a software perspective, the Jetson Nano runs a lightweight Ubuntu environment based on the **JetPack 5.1.2 SDK**, which provides direct support for CUDA, cuDNN, TensorRT, and OpenCV libraries. These NVIDIA components enable accelerated deep-learning inference directly on the device GPU, providing high throughput for YOLOv8 detection even under edge constraints.
The following points summarize the advantages of this embedded integration:

- **Low Latency:** On-device computation eliminates round-trip delay associated with cloud processing.
- **Compact Footprint:** The hardware, camera, and power supply together occupy less than 0.5 L of volume, making it deployable in small enclosures.
- **Energy Efficiency:** Despite continuous GPU usage, the system remains under 10 W average power draw.
- **Reliability:** Solid aluminum casing and passive cooling ensure uninterrupted operation over extended periods.
- **Scalability:** Multiple Jetson units can operate in parallel for large-scale installations such as parking lots or toll plazas.

Overall, this hardware architecture provides an ideal foundation for embedded computer-vision applications. Its modular design allows direct adaptation for related use cases such as *traffic violation detection*, *vehicle counting*, or *entry-exit automation*.

4 List Of Parts

5 Set Up

6 Functions

7 Maintenance

8 Safety Guidelines

Troubleshooting
Specifications

9 Troubleshooting

10 Add Ons

Bibliography

[Jai21] Jaided AI Research, *Easyocr: Ready-to-use ocr with 80+ languages supported*, Accessed October 2025, 2021. [Online]. Available: https://github.com/JaidedAI/EasyOCR.

[NVI24] NVIDIA Corporation, *Nvidia jetpack sdk 5.1.2 documentation*, Accessed October 2025, 2024. [Online]. Available: https://developer.nvidia.com/embedded/jetpack.

[PM23] A. Pathak and S. Mehta, "Optimizing edge ai inference on jetson devices using tensorrt and cuda," in *Proceedings of the IEEE Symposium on Embedded Vision Systems*, IEEE, 2023, pp. 112–118.

[Ult23] Ultralytics Team, "Yolov8: Next-generation object detection and segmentation model," *Ultralytics Technical Report*, 2023, Accessed October 2025. [Online]. Available: https://docs.ultralytics.com.